

Faculty of Engineering & Technology

Department of Electrical & Computer Engineering

ENCS3390: Operating Systems

Second Semester, 2025/2026

Project 1 Report

Prepared by:

Name: Abdalrahman Abdullah Number: 1231477

Name: Rahaf Mahmoud Number: 1232633

Instructor: Yazan Abu Farha

Section: 1 & 2

Date: 5-5-2026

Abstract

The objective of this project is to implement and compare multiple multithreaded strategies for matrix multiplication, including a baseline single-threaded approach, row-wise parallelism, column-wise parallelism, and block-based multiplication. These approaches are applied to large matrices in order to evaluate their performance and scalability on multi-core systems. The project aims to analyze how different work partitioning techniques affect execution time and overall efficiency. Experiments are conducted using various matrix sizes and thread counts to measure performance improvements and compute speedup. The results provide practical insights into parallel computing concepts such as concurrency, load balancing, and memory access patterns, highlighting the advantages and limitations of each approach.

Contents

Abstract I
List of Tables III
1- Describe the Environment of the computer: 4
2- How we achieved Multithreading Requirements: 5
3- An analysis according to Amdahl's law 6
4- A table that compares the Performance Comparison of the Approaches 7
5- Discussion and Comparison of the Four Approaches 8
6- Discussion of issues affecting performance 9
7- Conclusion 9

List of Tables

Table 1: Speedup Results 7
Table 2: Comparison of the Performance Comparison of the Approaches 7

1- Describe the Environment of the computer:

All experiments were performed on our personal laptops with the following specifications:

Hardware

- CPU: 13th Gen Intel Core i7-13700H
- Base Frequency: 3.70 GHz
- Max Turbo: up to 5.0 GHz
- Cores: 14 physical cores, where 8 are Efficient and the remaining 6 are Performance-cores.
- Threads: 20 Logical processors
- Cache: 24 MB Intel® Smart Cache

Memory

- RAM: 16 GB DDR5

Operating System (OS):

- Edition Windows 11 Pro
- Version: 25 H2
- OS build: 26200.8246
- Experience: Windows Feature Experience Pack 1000.26100.297.0

Programming Language and IDE

For this project, I used the C programming language because it is fast and gives good control over memory, which is important for matrix multiplication. It also supports multithreading using the pthreads library, which allowed me to implement different parallel methods.

I used the GCC compiler and ran the program in a terminal environment. This setup made it easy to compile the code with threads support and measure the execution time of each method.

Virtual Machine:

A virtual machine was not used in this project. All programs were run directly on the host operating system (Windows 11) **using a compatible environment.** This is because the project is written in C and uses pthreads, which work well on the host system without needing a virtual environment. Running directly on the machine also gives more accurate timing and performance results, since there is no extra overhead from virtualization.

2- How we achieved Multithreading Requirements:

In this project, multithreading was implemented using the pthread library in C to improve the performance of matrix multiplication. Instead of executing the entire computation sequentially, the workload was divided among multiple threads, allowing parallel execution on multi-core systems.

The matrices A, B, and C were declared as global arrays, which allows all threads to access them directly. Since threads share the same memory space within the same process, there was no need for additional communication or data copying, which reduces overhead and improves efficiency.

Thread management was handled using two main functions:

pthread_create (): used to create and start each thread. Every thread is assigned a specific portion of the computation based on the chosen parallelization strategy.

pthread_join (): used to ensure that the main program waits until all threads finish execution before measuring the total execution time.

In the multithreaded implementation, the matrix multiplication workload was divided among threads using different strategies. In one approach, each thread was assigned a subset of rows, where it computes all columns for its assigned rows, ensuring full coverage of the matrix. In another approach, each thread was assigned a subset of columns, where it computes all rows for its assigned columns. Additionally, the matrix was divided into smaller blocks (tiles), and each thread was responsible for computing one or more blocks, allowing balanced workload distribution.
In all approaches, each thread computes its assigned portion using the standard matrix multiplication algorithm. To ensure correctness and avoid race conditions, each thread writes to a unique portion of the result matrix, so no overlap occurs between threads.

Execution time was measured using QueryPerformanceCounter () for all implementations. This function measures real wall-clock time, meaning it records the actual time from when the threads start until the last thread finishes. Unlike clock () which accumulates CPU time across all threads and gives inflated results in multithreaded programs, QueryPerformanceCounter () gives an accurate reading of how long the program took to run, making the comparison between different strategies fair and reliable.

The program was tested using different numbers of threads, such as 2, 8, and 16, to evaluate scalability and measure performance improvements compared to the baseline sequential implementation.

3- An analysis according to Amdahl's law

1- What percentage is the serial part of your code?

The serial portion was estimated by comparing the baseline (single-threaded) execution time with the best multithreaded execution time recorded across all three strategies (row-wise, column-wise, and block). The difference between the two represents the serial part of the program, which includes thread creation, joining, and any non-parallelizable initialization.

We used the 1000x1000 matrix results since they give the most meaningful comparison.

Total Execution Time: 3.698993 s
Parallel Portion Time: 0.356540 s

Now for the percentage of the serial part

Speedup = T baseline / T parallel = 3.698993 / 0.356540 ≈ 10.37

Using Amdahl's Law, the serial fraction can be approximated as:

Speedup(N) = 1 / (S + (1 − S)/N)
Assuming N = 16 threads, the serial fraction can be estimated by solving the equation, which gives:

Serial ≈ 0.06 (≈ 6%)

Parallel ≈ 94%

This indicates that most of the program is parallelizable, and the remaining portion is due to overhead such as thread creation, synchronization, and memory access.

2- What is the maximum speedup according to the available number of cores?

Using the estimated serial fraction S ≈ 0.06 and Amdahl's Law:

Speedup(N) = 1 / (S + (1 − S)/N)

The system used has 14 cores (6 performance cores and 8 efficient cores) with 20 hardware threads.

For example, using 6 performance cores:

Speedup (6) ≈ 4.61×

For 16 threads:

Speedup (16) ≈ 8.41×

This shows that the achievable speedup is limited by the serial portion and system overhead. Although increasing the number of threads improves performance, the improvement is not linear due to factors such as synchronization overhead and memory access limitations.

And to calculate the Speedup Using the number of the Threads used, here's the result.

Table 1: Speedup Results

Thread num.	2	8	16
Speedup	3.699x	9.2244x	10.7612x

3- What is the optimal Number of Child Processes or threads?

The system used in this experiment has 14 cores (6 performance cores and 8 efficient cores) with 20 total hardware threads.

In practice, the optimal number of threads is determined experimentally. While increasing threads improves performance, using more threads than available cores may lead to diminishing returns due to context switching and memory contention.

Based on typical behavior, the best performance is usually achieved when the number of threads is close to the available hardware threads, but the exact optimal value depends on the measured execution times.

4- A table that compares the Performance Comparison of the Approaches

The following table summarizes the execution times and speedup values of the three implementations (row-wise, column-wise, and block/tiled) for different matrix sizes and thread counts (2, 8, and 16). These results are used to compare the performance of each parallelization strategy against the baseline implementation.

Table 2: Comparison of the Performance Comparison of the Approaches

Size	Baseline (s)	Row-wise						Column-wise						Block (Tiled)					
		2 T		8 T		16 T		2 T		8 T		16 T		2 T		8 T		16 T	
		t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp	t(s)	Sp
100x100	0.0025	0.003464	0.72	0.002201	1.14	0.001918	1.30	0.004	0.62	0.003	0.83	0.002	1.25	0.006	0.42	0.0045	0.56	0.003	0.83
500x500	0.266	0.14285	1.86	0.0698899	3.81	0.04869	5.46	0.141	1.89	0.063	4.22	0.045	5.91	0.129	2.06	0.079	3.37	0.069	3.86
1000x1000	3.69893	1.5238	2.43	0.450359	8.21	0.3599	10.28	1.446	2.56	0.446	8.29	0.3569	10.36	0.99998	3.70	0.401	9.22	0.335546	11.02

T = Threads t(s) = Time in seconds Sp = Speedup (Baseline / Time) All times averaged over 5 runs

5- Discussion and Comparison of the Four Approaches

1. Row-wise Discussion

The row-wise parallelization approach shows noticeable improvement as the matrix size increases. For small matrices (100×100), the speedup is limited due to parallel overhead, resulting in only slight performance gains. As the matrix size grows to 500×500 and 1000×1000, the speedup increases significantly, reaching up to around 10× with 16 threads. This indicates that row-wise decomposition benefits from workload distribution across threads, but its performance is still limited compared to other approaches due to less efficient cache utilization.

2. Column-wise Discussion

The column-wise approach demonstrates better scalability compared to the row-wise method, especially for medium and large matrix sizes. While performance at 100×100 is still affected by overhead, at 500×500 and 1000×1000 it achieves higher speedup, slightly outperforming row-wise execution. This improvement is likely due to better memory access patterns compared to row-wise, although it still does not fully optimize cache usage. Overall, column-wise parallelization provides a balanced improvement in performance.

3. Block Discussion

The block approach provides the best performance for large matrix sizes. Although it performs poorly for small matrices (100×100) due to high overhead and complexity, its advantage becomes clear as the matrix size increases. At 1000×1000, it achieves the highest speedup, outperforming both row-wise and column-wise methods. This is mainly due to improved cache locality, as data is processed in smaller blocks that fit better in cache memory. Therefore, block-based parallelization is the most efficient approach for large-scale computations.

6- Discussion of issues affecting performance

- **Thread Creation Overhead** For small matrices like 100×100, the time spent creating and managing threads exceeded the actual computation time, causing nearly all parallel methods to perform worse than the baseline. This overhead becomes negligible as matrix size grows.

- **Memory Access Patterns** C stores arrays in row-major order, making row-wise access more cache-friendly than column-wise. In practice the difference was not significant in our results, likely because the matrices fit well within cache at the tested sizes.

- **Testing Environment** Running on Windows with a MinGW pthread implementation rather than native Linux threads may have reduced the actual parallelism achieved and contributed to lower-than-expected speedups, particularly at smaller matrix sizes.

- **Timing Accuracy** The clock () function measures accumulated CPU time across all threads rather than real elapsed wall-clock time, which can produce misleading values in multithreaded programs and was a source of inaccuracy in some of our measurements.

7- Conclusion

This project implemented and compared four approaches to matrix multiplication: single-threaded baseline, row-wise, column-wise, and block (tiled) parallelism, tested across matrix sizes of 100×100, 500×500, and 1000×1000 using 2, 8, and 16 threads.

The results confirm that parallelism provides little benefit for small matrices. At 100×100, all parallel methods performed worse than the baseline due to thread creation overhead. At 1000×1000 with 16 threads, all three methods exceeded 10× speedup, with block tiled achieving the best result at 11.02×, followed by column-wise at 10.36× and row-wise at 10.28×. The differences between the three strategies are small at large scale, but block multiplication has a consistent edge due to better cache locality.

Overall, the results confirm that the benefit of multithreading grows with problem size, and that both the choice of partitioning strategy and thread count play an important role in achieving optimal performance.